

Group Project - COMP 262 Section 401 Winter 2026

Engineering Technology & Applied Science, Centennial College

Natural Language & Recommender Systems

Students:

Mahyar Eslami

Mauli Gandhi

Rouhollah Kamali

Jose Pieruccini Arias

Arbina Shemael

Professor:

Sajid Hussain

Table of Contents

Phase 1.....	4
Dataset Exploration	4
Dataset pre-processing.....	5
Models and Assumptions.....	6
Model Evaluation and Comparison	6
5. Model Explanation.....	7
5.1 Lexicon-based Sentiment Analysis Overview	7
5.2 Model 1: VADER (Valence Aware Dictionary and Sentiment Reasoner)	7
How VADER Works	7
How We Convert VADER Score into Labels.....	8
5.3 Model 2: TextBlob	8
How TextBlob Works	8
How We Convert TextBlob Score into Labels	8
5.4 Decision Thresholds and Label Mapping.....	9
5.5 Why Macro vs Weighted Metrics Matter (Imbalanced Dataset)	9
6. Fine Tuning.....	9
6.1 Threshold Tuning	10
6.2 Sampling Strategy (Stratified Sample of 1000)	10
6.3 Preprocessing Tuning Per Lexicon	10
6.4 What Was Not Tuned (And why)	10
7. Testing Results Summary.....	11
7.1 Evaluation Metrics Used	11
7.2 Confusion Matrix – VADER.....	11
7.3 Confusion Matrix – TextBlob	12
7.4 Performance Comparison Table.....	13
7.5 score distributions & agreement analysis.....	14
7.6 Error Analysis (Misclassified Examples & Patterns).....	15
8. Conclusion	15
9. References	16

10. Appendix 1 – Project Plan.....	16
11. Appendix 2 – Meeting Register.....	16
Phase 2.....	18
Phase 2 Dataset Exploration	18
Text pre-processing.....	18
Review Summarization using LLM	21
Sample Results (First Two Reviews).....	22
Example 1	22
Example 2	22
Observations and Analysis	22
Automatic Customer Service Response Generation	23
Sample Result.....	23
Observations and Analysis	24
Combined Conclusion for LLM Tasks	24

Phase 1

For this initial phase our team focused on building a lexicon-based sentiment analysis pipeline. We downloaded the Industrial and Scientific reviews from amazon. This dataset was explored, preprocessed and analyzed to complete this part of the assignment. The corpus will help classify sentiment through VADER and TextBlob.

Dataset Exploration

After importing the dataset, we conducted some data exploration and discovered that the k-core review database contains 1.758,333 reviews in total, 165,764 products were reviewed and we identified 1,246,131 unique user accounts. The type of data found in this set is mostly text, but there are also summaries, star ratings, time stamps and some identifiers. Table 1 summarizes the basis statistics

Table 1. Basis Statistics of Amazon Industrial and Scientific Reviews Dataset

<i>Description</i>	<i>Total</i>	<i>Description</i>	<i>Total</i>
Reviews	1.758,33	Average Review Length	184.85
Unique Products	165,764	Median Review Length	94
Unique Users	1,246,131	Average Word Count	34.27
Average Rating	4.29	Median Words/Review	17
Median Rating	5	Maximum Words/Review	5,966
Std. Deviation	1.24	Lengthy Outliers	8.45%

We also noticed the dataset is overall quite positive, since **67.9%** of all reviews had a **5 star** rating, that explains why the average is at 4.29. Unfortunately, this means the set is heavily imbalanced. 4 stars reviews represent 13.7% of the dataset, while only 6.26% account for 3 stars. The lowest group is the 2 stars review with close to 4.1% and 1 star reviews are the third most common rating. Based on IQR (Q1=6, Q3=39) those reviews longer than ~88 words were considered lengthy outliers, that's roughly 148,564 or 8.45%.

Reviews Per Products

On average each product got 10.61 reviews with a median of 2, there were only 78,651 products with 1 review and the most reviewed item had 14,331 reviews. This could suggest a long tail distribution, because some products dominate the bulk of reviews across the board.

Each user contributed an average of 1.41 reviews with a median of 1. This also means 78% of unique users had only one review (983,487 out of 1,246,131). The most active user entered 156 reviews.

Missing Values

Very few came with images, because images were missing for 98% of all reviews, similar to vote with 88% missing. Style was missing on 60% of reviews. ReviewText and summary did not even reach a 1% missing from all reviews.

We spotted some duplicates, 377,093 were duplicate texts and 87,081 were fully duplicated rows. Is possible that these duplicates came from syndicated reviews or potentially from repeated purchases. Nevertheless, the dataset proved to be highly imbalanced, since the majority of reviews are positive; length of reviews is generally short is a small fraction of long outliers, average reviews per customer did not even hit 1.5. It is convenient that the missing values do not impact the sentiment analysis.

Dataset pre-processing

We kept '*reviewText*' since it's the primary source of sentiment in the set, we noticed '*summary*' can provide sentiment cues, therefore we decided to maintain it as well. '*Overall*' tends to derive sentiment labels, so we will use it; '*asin*' is needed for product level analysis and '*reviewerID*' for the same reason at a customer or individual level. The rest of the fields were deleted since they were not relevant to the project or had significant missing entries. As per professor instructions we are classifying Positive ratings as either 4 or 5 stars, 3 star reviews are treated as neutral and 1-2 means negative review.

Outliers did not even represent 10% of the dataset and we kept them because lexicon models can handle such long reviews, likewise we did not want the distribution to be distorted.

VADER: it required minimal cleaning, retained capitalization, punctuation and key intensifiers. This lexicon is important

TextBlob: helped with whitespace normalization and handled internal tokenization.

We created a new field “full text” which combines summary and review Text. Merging these two features could give us more clear signals about customers preferences and ultimately their sentiment.

Models and Assumptions

Lexicon Models

We used VADER, from package Vader Sentiment which is a Lexicon + rule-based model. Assumptions for this model include: Caps increase intensity, same thing for repeated punctuation (!!!). It also detects degree modifiers such as “very” or “extremely” since they tend to adjust polarity. Negations are also recognized and the compound score $\in [-1,1]$ is supposed to help with classification. Although VADER is simple to use and interpret, it might misclassify some edge instances like sarcastic comments, ambiguous feelings etc.

We did not select **SentiWordNet** due to its heavy processing requirements, specialized tagging and lemmatization. Overall, it did not seem to be the best fit for informal product reviews.

Model Evaluation and Comparison

We used stratified sample of one thousand reviews, we evaluated both models on accuracy, precision, recall and F1-Score:

<i>Metric</i>	<i>VADER</i>	<i>TextBlob</i>	<i>Better Model</i>
<i>Accuracy</i>	0.793	0.735	VADER
<i>Macro Precision</i>	0.457	0.46	TextBlob
<i>Macro Recall</i>	0.514	0.45	VADER
<i>Macro F1</i>	0.47	0.43	VADER
<i>Weighted F1</i>	0.82	0.78	VADER

In this case the Macro F1 is the most appropriate measure since we have to account for class imbalance. Overall VADER slightly outperforms TextBlob, on Macro F1 it indicates a better handling of all classes. This could be because TextBlob is usually more conservative and often predicts neutral. VADER was better at capturing informal cues and emphasis words, quite common in this Amazon review environment.

Error Analysis

TextBlob misclassified **26.5%** of the samples while VADER only **20.7%**. Potentially due to sarcasm, mixed sentiment for statements like “Loved the product but had a terrible shipping experience”; short reviews might have contributed as well and the neutral sentences which are so challenging.

5. Model Explanation

5.1 Lexicon-based Sentiment Analysis Overview

In Phase 1, sentiment classification is performed using a lexicon-based approach, where sentiment is determined from a predefined dictionary of words with associated polarity scores rather than learning patterns from training data. Each review is scored, and the score is mapped into one of three classes: Positive, Neutral, or Negative. This approach is useful in early-stage NLP projects because it provides a fast baseline without requiring model training.

A key limitation is that lexicon methods often struggle with sarcasm, domain-specific language, and mixed sentiment sentences because they evaluate words and simple rules rather than deeper context.

5.2 Model 1: VADER (Valence Aware Dictionary and Sentiment Reasoner)

How VADER Works

VADER is a rule-based sentiment analyzer designed for short, informal text. It uses:

- A sentiment lexicon (dictionary) where words have polarity strength.

- Rules that adjust polarity using punctuation, capitalization, degree modifiers (e.g., “very”), and negations.

VADER outputs four scores:

- positive, neutral, negative (proportions)
- compound score: a normalized value between -1 and +1 summarizing overall sentiment.

How We Convert VADER Score into Labels

Using the compound score:

- Positive if compound ≥ 0.05
Negative if compound ≤ -0.05
- Neutral otherwise

This thresholding method is a standard practice for VADER and produces a clear 3-class mapping.

5.3 Model 2: TextBlob

How TextBlob Works

TextBlob provides sentiment using a polarity score derived from an underlying lexicon-based mechanism. It returns:

- polarity in the range $[-1, +1]$
- subjectivity (not used for classification here)

TextBlob is simple to use and often provides a stable baseline, but it does not handle punctuation emphasis/capitalization as strongly as VADER.

How We Convert TextBlob Score into Labels

Using polarity:

- Positive if polarity > 0.1
- Negative if polarity < -0.1
- Neutral otherwise

These thresholds make neutral classification less sensitive (more conservative), reducing over-classification into Positive/Negative for slightly emotional text.

5.4 Decision Thresholds and Label Mapping

Both models use thresholding to convert continuous sentiment scores into classes.

Why thresholding is needed:

Lexicon tools produce a real-valued score, but the task requires categorical prediction (Positive/Neutral/Negative). Thresholds also reduce noise from weak sentiment signals.

Ground truth label mapping (from ratings):

- Ratings 4–5 → Positive
- Rating 3 → Neutral

This allows us to compare lexicon predictions against “true labels” derived from ratings.

5.5 Why Macro vs Weighted Metrics Matter (Imbalanced Dataset)

Because the dataset is heavily skewed toward Positive reviews, accuracy alone can appear high even if a model performs poorly on Neutral and Negative. Therefore:

- Macro-averaged precision/recall/F1 treats each class equally and is fairer for imbalanced datasets.
- Weighted-average metrics reflect class proportions and can be inflated by the majority class.

This is why the report treats Macro F1 as the primary metric for comparing VADER and TextBlob.

6. Fine Tuning

Lexicon models do not train parameters like machine learning classifiers, but we can still “fine tune” decisions that affect performance.

6.1 Threshold Tuning

Fine tuning in this phase focuses mainly on selecting decision boundaries:

- VADER: compound thresholds at ± 0.05
- TextBlob: polarity thresholds at ± 0.1

These thresholds are chosen to balance sensitivity and avoid forcing weak sentiment into Positive/Negative classes.

6.2 Sampling Strategy (Stratified Sample of 1000)

Instead of testing on the full dataset, the pipeline selects 1000 reviews for evaluation. To ensure minority classes are represented fairly, the sample is taken in a stratified manner by sentiment label (so the sample mirrors class distribution, and every class is included).

This improves reproducibility and makes model comparison manageable while still statistically meaningful.

6.3 Preprocessing Tuning Per Lexicon

The preprocessing is intentionally minimal:

- VADER: minimal preprocessing to preserve punctuation, capitalization, and modifiers.
- TextBlob: light cleaning (HTML removal + whitespace normalization).

Heavy preprocessing such as stop word removal or stemming is avoided because these lexicons rely on surface cues that can be removed by aggressive cleaning.

6.4 What Was Not Tuned (And why)

No learned weights or training epochs exist for lexicon models. Therefore:

- No hyperparameter search (GridSearch/RandomSearch) was used.
- No model fitting/training was performed.

- Fine tuning is limited to thresholds, sampling methods, and preprocessing choices.

7. Testing Results Summary

7.1 Evaluation Metrics Used

Models were evaluated using:

- Accuracy
- Precision / Recall / F1 (Macro)
- Precision / Recall / F1 (Weighted)
- Confusion Matrix
- Agreement % (VADER vs TextBlob)
- Score distribution plots
- Error analysis (misclassified review patterns)

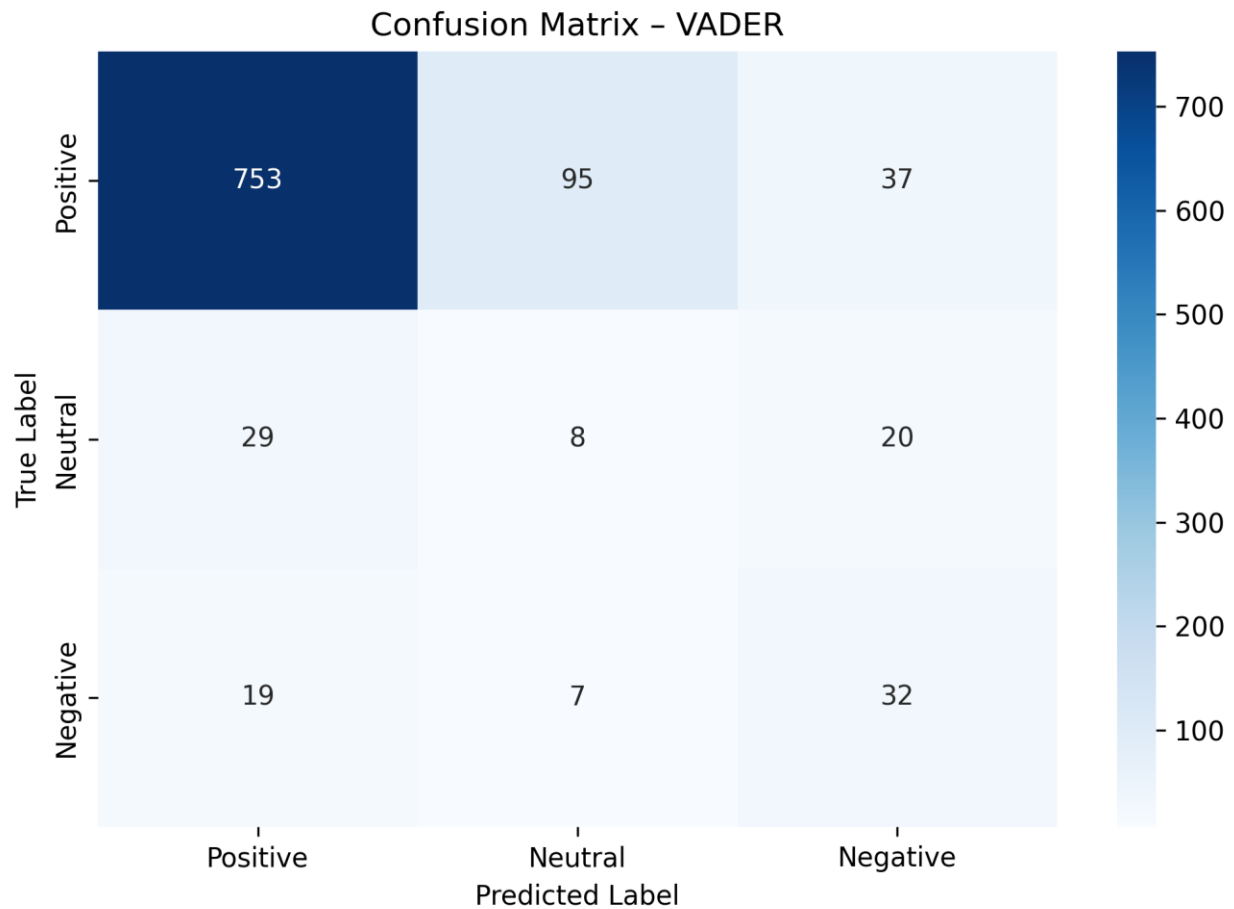
These give a complete view of performance, especially under class imbalance.

7.2 Confusion Matrix – VADER

What it shows: how many reviews from each true class were predicted as Positive/Neutral/Negative.

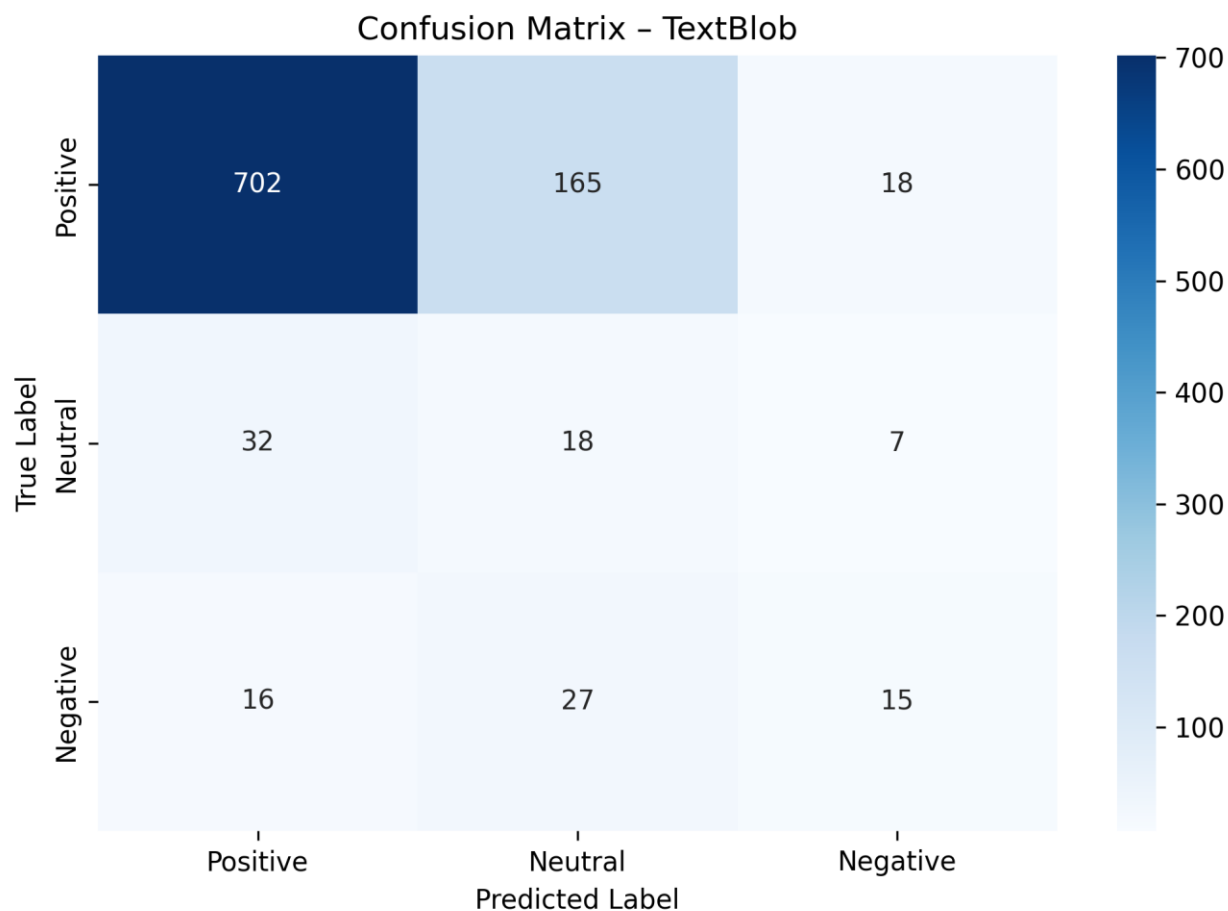
Key observation:

VADER performs strongly on the Positive class (majority), but like most lexicon methods it has difficulty separating Neutral from weak Positive/Negative, leading to confusion near the threshold boundaries.



7.3 Confusion Matrix – TextBlob

TextBlob also predicts Positive well, but tends to classify more cases as Neutral compared to VADER. This can reduce false positives but may also reduce recall for Positive and Negative depending on the text style.



7.4 Performance Comparison Table

The comparison table summarizes results across all metrics. The key finding is that **VADER outperforms TextBlob on Macro F1**, which is the fairest indicator for this dataset.

	A	B	C	D
1	Metric	VADER	TextBlob	Difference (V-T)
2	Accuracy	0.793	0.735	0.058
3	Precision (macro)	0.45745091363068885	0.4655714285714286	-0.008120515
4	Recall (macro)	0.5143074909170452	0.455876834	0.058430656809567216
5	F1-Score (macro)	0.4748069890190327	0.4332231686613415	0.041583820357691204
6	Precision (weighted)	0.8569656792645557	0.8549957142857143	0.001969965
7	Recall (weighted)	0.793	0.735	0.058
8	F1-Score (weighted)	0.8212287927594589	0.7854037980515369	0.035824995
9				

This visually compares metrics and includes agreement results.

7.5 score distributions & agreement analysis

Two important visual analyses were produced:

1. VADER score distribution by true label
Shows how Positive/Neutral/Negative reviews spread across compound score values.
2. TextBlob polarity distribution by true label
Shows how polarity values overlap between classes (especially Neutral overlap).
3. Agreement pie chart
Measures how often both models produce the same label. Agreement is high (around ~80%), showing both lexicons behave similarly overall, but differ mainly around Neutral boundaries.



7.6 Error Analysis (Misclassified Examples & Patterns)

Misclassifications were inspected to find common failure patterns. Lexicon methods typically fail when:

- The review contains sarcasm/irony (“great... not”)
- The review has mixed sentiment (“good product but terrible shipping”)
- The review is too short or vague (“ok”, “fine”)
- The review is Neutral but contains weak positive/negative words

This explains why macro metrics are lower: Neutral and Negative are harder classes in real-world review text.

8. Conclusion

In Phase 1, two lexicon-based sentiment models (VADER and TextBlob) were implemented and evaluated on the Amazon Industrial & Scientific review dataset. Overall, **VADER achieved better performance than TextBlob**, especially on macro-averaged metrics, indicating stronger handling of minority classes compared to TextBlob.

However, both lexicon methods show limitations on ambiguous and Neutral reviews and cannot learn domain-specific sentiment the way machine learning approaches can. Phase 2 will improve results by applying supervised learning models trained on labeled reviews.

9. References

1. Ni, J., Li, J., & McAuley, J. (2019). *Justifying recommendations using distantly-labeled reviews and fine-grained aspects*. (Amazon Review Datasets).
2. VADER Sentiment (Hutto & Gilbert). VADER GitHub repository / documentation.
3. TextBlob Documentation.
4. Scikit-learn documentation (metrics and confusion matrix).

10. Appendix 1 – Project Plan

Week	Tasks	Deliverable
1	Dataset selection and data exploration	Exploration summary and visualizations
2	Data preprocessing and cleaning	Preprocessed dataset file
3	Lexicon-based sentiment analysis	Sentiment results and confusion matrices
4	Model comparison and evaluation	Model comparison report and table
5	Visualization refinement and interpretation	Final plots and analysis section
6	Final report writing and compilation	Final consolidated report

11. Appendix 2 – Meeting Register

Date	Time	Topics Discussed	Action Items
2026-01-15	8–9PM	Dataset selection and task division	Finalize dataset and assign sections

2026-01-22	8–9PM	Data exploration review	Prepare visualizations
2026-01-29	8–9PM	Preprocessing discussion	Clean and prepare dataset
2026-02-05	8–9PM	Sentiment model discussion	Run lexicon models
2026-02-12	8–9PM	Model comparison review	Prepare evaluation results
2026-02-19	8–9PM	Final review and edits	Compile final report

Phase 2

In phase two we extend the work we completed on the previous phase, which included the implementation and evaluation of two lexicon-based sentiment models (VADER and TextBlob). For this second part of the assignment, we worked on AI based sentiment classifiers; and we compared them with their lexicon models. In addition, we constructed a sentiment enhanced recommender system and we ran locally hosted Hugging Face LLMs to execute summarization and customer service answer generation. At the beginning we used the Amazon Industrial & Scientific reviews set, briefly explored on the first stage of this project. This time around we went with a larger subset (≥ 2000 reviews) to train our machine learning models, since it was specified for this assignment

Phase 2 Dataset Exploration

We imported a subset of more than 2,000 reviews from the full Industrial & Scientific dataset. For the exploration process we continued the same style, however, this time we focus on the subset we used for machine learning training. We noticed the length of reviews; their distributions and missing values were consistent with the findings and observations from phase 1.

The set remained highly imbalanced as well, because predominantly these reviews were rated 4-5 stars, with 3 stars (neutral), 1-2 stars being the minority classes in the set. This just reinforced the need for using a stratified splitting technique as well as macro-averaged metrics.

Text pre-processing

Lexicon models are different from ML models since ML classifiers primarily rely on statistic patterns instead of predefined dictionaries, keeping this in mind we performed lowercasing algorithms, removed the punctuation and html artifacts, applied some tokenization, TD-IDF vectorization and we combined the summary feature with review Text just like we did on the previous phase.

We took those steps because TF-IDF needs normalized text to reduce noise, also VADER, machine learning models cannot rely on capitalization for intensify cues, same thing with punctuation. We thought merging summary and review Text could provide solid signal density.

Text Representation

We decided to go with TF-IDF for the representation method since it performs well on sparse and high dimensional text. It's also able to capture domain specific vocabulary such as "defect", "broke" etc. It seems to work efficiently with linear models including Logistic Regressions and SVM, we also think it will be computationally efficient when dealing with this large dataset.

Machine Learning Models

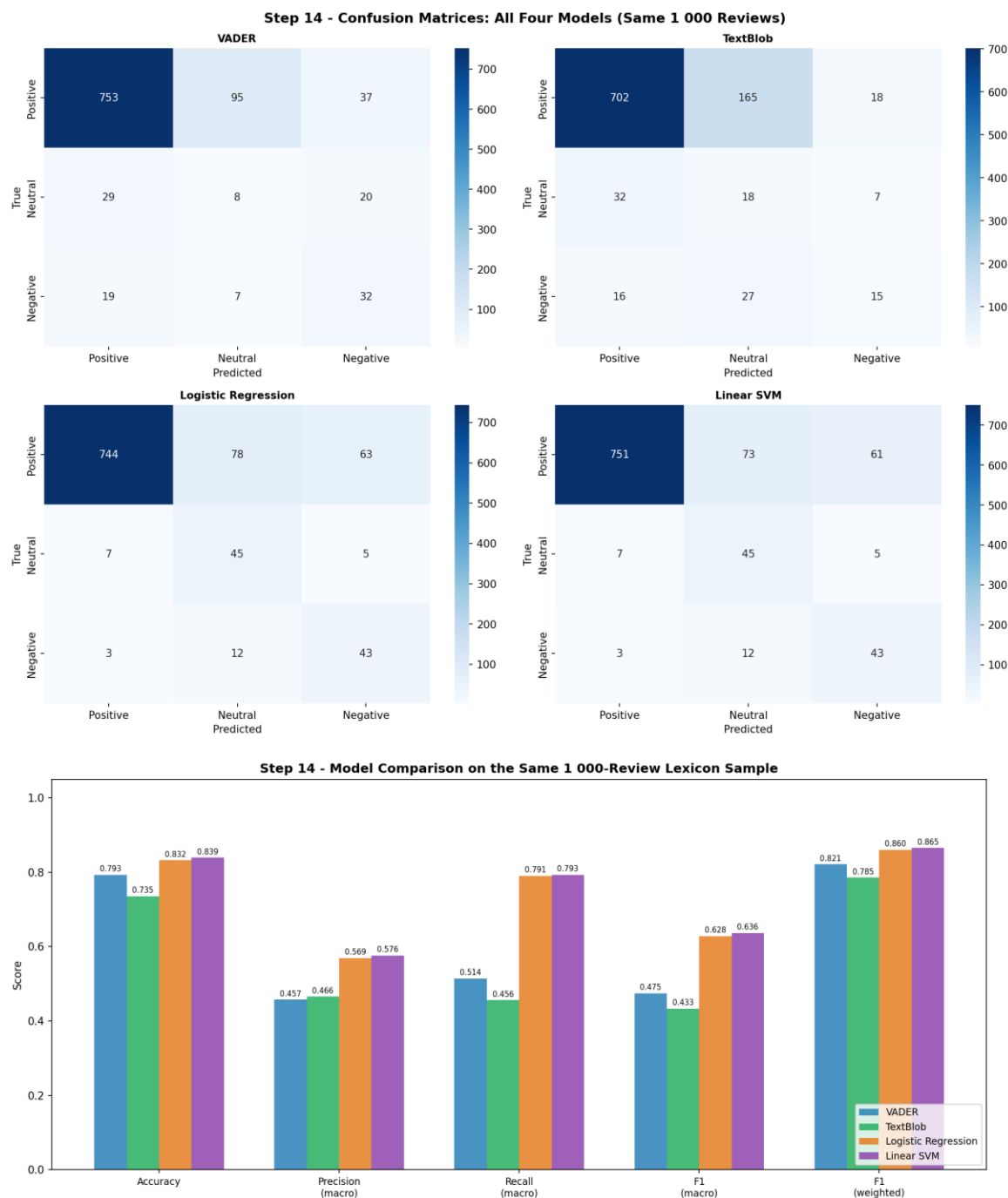
We ran two models, one logistic regression and a linear support vector machine (SVM), both were trained with 70% of the entire set using stratified sampling and we set aside 30% for testing. We also tuned hyperparameters by running grid search. For efficiency reasons we limited TF-IDF features to a maximum vocabulary size. We tackled the class imbalance through the implied stratification of data split.

Testing Results

We evaluated the accuracy of each model as well as their precision, recall and F1-Scores, we also produced some confusion matrices.

Model	Accuracy	F1 Macro	F1 Weighted
Linear SVM	0.8390	0.6363	0.8649
Logistic Regression	0.8320	0.6285	0.8597
VADER	0.7930	0.4748	0.8212
TextBlob	0.7350	0.4332	0.7854

Based on the results both machine learning models outperform lexicon models, and this happened across all metrics we evaluated. Linear SVM proved to be the best out of all four, due to the highest macro F1. We also noticed that neutral reviews continue to be the most difficult class, something not new to us, since we saw the same trend in phase 1.



Cross-Model Comparison (Lexicon vs ML)

To guarantee fairness in the assessment we tested all four models on the same 1,000 stratified sample of reviews we dealt with on phase 1. Some of the most important findings were how ML models kept outperforming lexicon models. Clearly lexicon models struggled with mixed sentiment and some specific domain specific terms. ML models tend to learn patterns like

“broke”, “defect” and "return" which could enhance negative class recall. Based on the results, Linear SVM is the ideal production model for further use in this project.

Recommender System: Sentiment Enhanced Collaborative Filtering

We enhanced rating values using review text and we implemented a Sentiment Enhanced Item which is based on collaborative filtering. Below is the enhancement formula:

$$\text{Sentiment_score} = \frac{\text{Vader_compound} + 1}{2}$$

$$\text{normalized_star} = \frac{\text{rating} - 1}{4}$$

$$\text{enhanced_rating} = 0.6 \cdot \text{normalized_star} + 0.4 \cdot \text{sentiment_score}$$

Results

System	MAE	RMSE	Coverage
Original CF	0.2445	0.7076	0.2426
Enhanced CF	0.2282	0.5316	0.2426

MAE seemed to improve by 6.7%, while RMSE went up by 24.9%. Overall sentiment helps minimize noise in sparse rating matrices. We can conclude that enhanced ratings with sentiment is able to deliver more accurate and stable recommendations, really helpful for controversial products.

Review Summarization using LLM

In this step, we implemented automatic summarization of customer reviews using a pre-trained transformer-based model from Hugging Face. The model selected was **sshleifer/distilbart-cnn-12-6**, which is a lightweight version of BART optimized for abstractive summarization tasks while maintaining good performance and efficiency.

We selected 10 reviews with length greater than 100 words to ensure sufficient textual content for meaningful summarization. Each review was passed through the model to generate a concise summary of approximately 50 words. The objective of this step is to reduce long customer reviews into shorter, more readable representations while preserving the key information.

The choice of this model is justified by its ability to perform **abstractive summarization**, meaning it generates new sentences rather than simply extracting parts of the original text. This allows the summaries to be more natural and easier to understand.

Sample Results (First Two Reviews)

Example 1

Original Review (excerpt, ~120 words):

“As another reviewer mentioned I don't think there is any difference in which Goop you buy... The only reason I dropped off a star is that it often hardens after a tube is opened...”

Generated Summary (38 words):

“The only reason I dropped off a star is that it often hardens after a tube is opened even though the cap is on tight. Other than that, the product itself is 5-stars all the way.”

Example 2

Original Review (excerpt, ~130 words):

“These wipes are true to their name in that they do not shred or leave lint traces... they work better than other towels or wipes because of how clean they leave the surface...”

Generated Summary (35 words):

“These wipes are true to their name in that they do not shred or leave lint traces. They work better than other towels or wipes because of how clean they leave the surface.”

Observations and Analysis

From the results, we observed that the model was able to effectively capture the **main sentiment and key aspects** of the reviews, such as product quality and issues, while significantly reducing the text length. The summaries are coherent and preserve the essential meaning of the original content.

However, some limitations were observed:

- Minor details and secondary information are sometimes omitted
- Summaries are occasionally shorter than the desired 50-word target
- The model tends to focus more on dominant sentiment rather than balanced viewpoints

Overall, the summarization approach proved to be effective for condensing long reviews into meaningful short text, which can be useful for quick analysis and large-scale review processing.

Automatic Customer Service Response Generation

In this step, we implemented automatic customer service response generation using a large language model. The model selected was **google/flan-t5-large**, an instruction-tuned transformer model capable of generating structured and context-aware responses.

To improve response quality, we first extracted the **main complaint** from the customer review using a keyword-based filtering approach. This ensures that the model focuses only on the relevant issue instead of processing the entire review text.

The extracted complaint was then embedded into a carefully designed prompt, instructing the model to generate a response as if it were a customer service representative. This prompt-based approach allows the model to generate polite, structured, and human-like replies.

Sample Result

Customer Review (excerpt):

“I’ve found all kinds of uses for this product... however when removing it, the adhesive pulled paint off the wall and is not wall-friendly...”

Extracted Complaint:

“The adhesive pulled paint off walls and is not wall-friendly like other products.”

Generated Customer Service Response:

“Dear Customer, thank you for taking the time to share your experience with product B00006IC2O. We sincerely apologize for the issue you described... We understand how frustrating this must be... Please contact our support team and we will assist you further.”

Observations and Analysis

From the results, we observed that the generated responses are:

- **Coherent and well-structured**
- **Polite and aligned with real customer service tone**
- Able to acknowledge customer concerns and suggest possible solutions

This demonstrates the capability of LLMs in automating customer support interactions.

However, some limitations were identified:

- Responses can be **generic and not product-specific**
- The model lacks real business context and cannot access order-level data
- Suggested solutions may not always be fully actionable in real scenarios

Despite these limitations, this approach shows strong potential for integrating LLMs into automated support systems and improving user experience.

Combined Conclusion for LLM Tasks

Both summarization and response generation demonstrate how modern NLP systems can extend beyond classification into **text generation tasks**. Summarization helps reduce large textual data into concise insights, while response generation enables interactive applications such as automated customer support.

However, both approaches rely on pre-trained models and lack domain-specific fine-tuning, which can impact accuracy and contextual relevance. Future improvements could include fine-tuning models on domain-specific datasets or integrating retrieval-based systems for more precise responses.